

Programação Orientada a Objetos

Gestão de hospital

Segunda Entrega

Trabalho Realizado por:

Daniela Brito – nº 25591

Ano Letivo 2023/2024

19 de dezembro de 2023

Índice

Introdução.....	3
Contexto	3
Relato Técnico do Projeto	4
Funcionalidades.....	5
1. Gestão de Pacientes.....	5
• Adicionar Paciente:	5
• Listar Pacientes:	5
2. Gestão de Médicos, Enfermeiros e Funcionários.....	5
• Adicionar Médico, Enfermeiro e Funcionário:	5
• Listar Médicos, Enfermeiros e Funcionário:.....	5
3. Agendamento e Gestão de Consultas	5
• Agendar Consulta:	5
• Listar Consulta:	6
4. Realização de Exames	6
• Realizar Exame:	6
5. Gestão Hospitalar Geral	6
• Informações do Hospital:	6
• Listar Staff:	6
Diagrama de Classes.....	7
Descrição das Classes	8
Classe Hospital	8
Classe Staff (Classe Abstrata)	9
Classe Paciente.....	9
Classe Consulta.....	10
Classe Exame	10
Classe Cama.....	11
Classe Diagnóstico	11
Classe Médico.....	12
Classe Enfermeiro.....	12
Classe Funcionário.....	13
Conclusão	14

Introdução

Este projeto representa um desafio significativo no âmbito do curso, concentrando-se na criação de um sistema de gestão para um hospital privado utilizando os princípios da Programação Orientada a Objetos (POO) e a linguagem C#. O objetivo principal é desenvolver uma plataforma que centralize e otimize as operações cruciais para o eficiente funcionamento da instituição de saúde.

Contexto

O hospital privado em destaque emerge como uma instituição comprometida em oferecer atendimento de excelência, integrando tecnologia de ponta e uma equipa altamente qualificada de profissionais de saúde. Com especialidades médicas diversas e serviços de diagnóstico abrangentes, o hospital destaca-se pela prestação de cuidados personalizados e abrangentes aos seus pacientes.

Ao longo deste relatório, irei detalhar a estrutura e organização dos dados, apresentar um diagrama de classes que reflete a arquitetura do sistema e fornecer descrições mais aprofundadas das classes envolvidas. A abordagem utilizada visa não apenas atender aos requisitos do projeto, mas também estabelecer um código modular e bem-organizado, garantindo flexibilidade e facilitando a manutenção futura.

Relato Técnico do Projeto

Desde o início, o meu foco foi desenvolver um código estruturado e modular, seguindo as melhores práticas de desenvolvimento de software. Utilizando o conceito de Programação Orientada a Objetos (POO), as classes foram projetadas para refletir entidades do mundo real, proporcionando uma representação clara e eficaz dos elementos fundamentais do hospital, como pacientes, médicos, funcionários e consultas.

Cada classe foi cuidadosamente elaborada para desempenhar um papel específico no sistema, promovendo uma organização coesa e facilitando futuras expansões. A transição de *arrays* para listas, como sugerido na segunda fase, proporcionou uma maior flexibilidade no gerenciamento dinâmico dos dados, alinhando-se com as boas práticas modernas.

Os desafios encontrados durante a implementação incluíram a adaptação do código às diretrizes do professor, a compreensão detalhada das relações entre as entidades e a manutenção de um código limpo e legível. A introdução de *Regions* e uma documentação adequada contribuíram para uma melhor organização e compreensão do código.

Ao refletir sobre o processo de desenvolvimento até agora, percebo a importância de considerar não apenas a funcionalidade do código, mas também a sua manutenibilidade e escalabilidade. A utilização de *Regions* e uma documentação clara tornou-se uma prática valiosa, proporcionando uma visão mais estruturada do código.

Em resumo, este relatório fornecerá uma análise abrangente da estrutura do código, apresentando um diagrama de classes para ilustrar a arquitetura do sistema e fornecendo descrições detalhadas das classes envolvidas. O objetivo é não apenas atender aos requisitos do projeto, mas também estabelecer uma base sólida para o desenvolvimento futuro e promover uma compreensão clara do código implementado.

Funcionalidades

1. Gestão de Pacientes

- **Adicionar Paciente:**

Permite a inserção de um novo paciente no sistema, fornecendo um meio eficiente de registar informações essenciais sobre os indivíduos atendidos no hospital.

- **Listar Pacientes:**

Oferece uma visão detalhada de todos os pacientes cadastrados, facilitando a identificação e acompanhamento dos casos médicos.

2. Gestão de Médicos, Enfermeiros e Funcionários

- **Adicionar Médico, Enfermeiro e Funcionário:**

Permite a inclusão de novos profissionais de saúde e funcionários, com a especificação de detalhes como categoria, especialidade e setor de atuação.

- **Listar Médicos, Enfermeiros e Funcionário:**

Fornece uma lista organizada dos profissionais e funcionários cadastrados, facilitando o acompanhamento da equipe e a gestão de recursos humanos.

3. Agendamento e Gestão de Consultas

- **Agendar Consulta:**

Possibilita a marcação de consultas, associando pacientes e médicos, com a definição da data para melhor organização e acompanhamento dos atendimentos.

- **Listar Consulta:**

Apresenta uma visão geral de todas as consultas agendadas, incluindo informações relevantes sobre pacientes, médicos e datas, para uma gestão eficaz do fluxo de atendimentos.

4. Realização de Exames

- **Realizar Exame:**

Oferece a funcionalidade de realizar exames em pacientes, sendo uma extensão do papel dos profissionais de saúde para diagnósticos mais precisos.

5. Gestão Hospitalar Geral

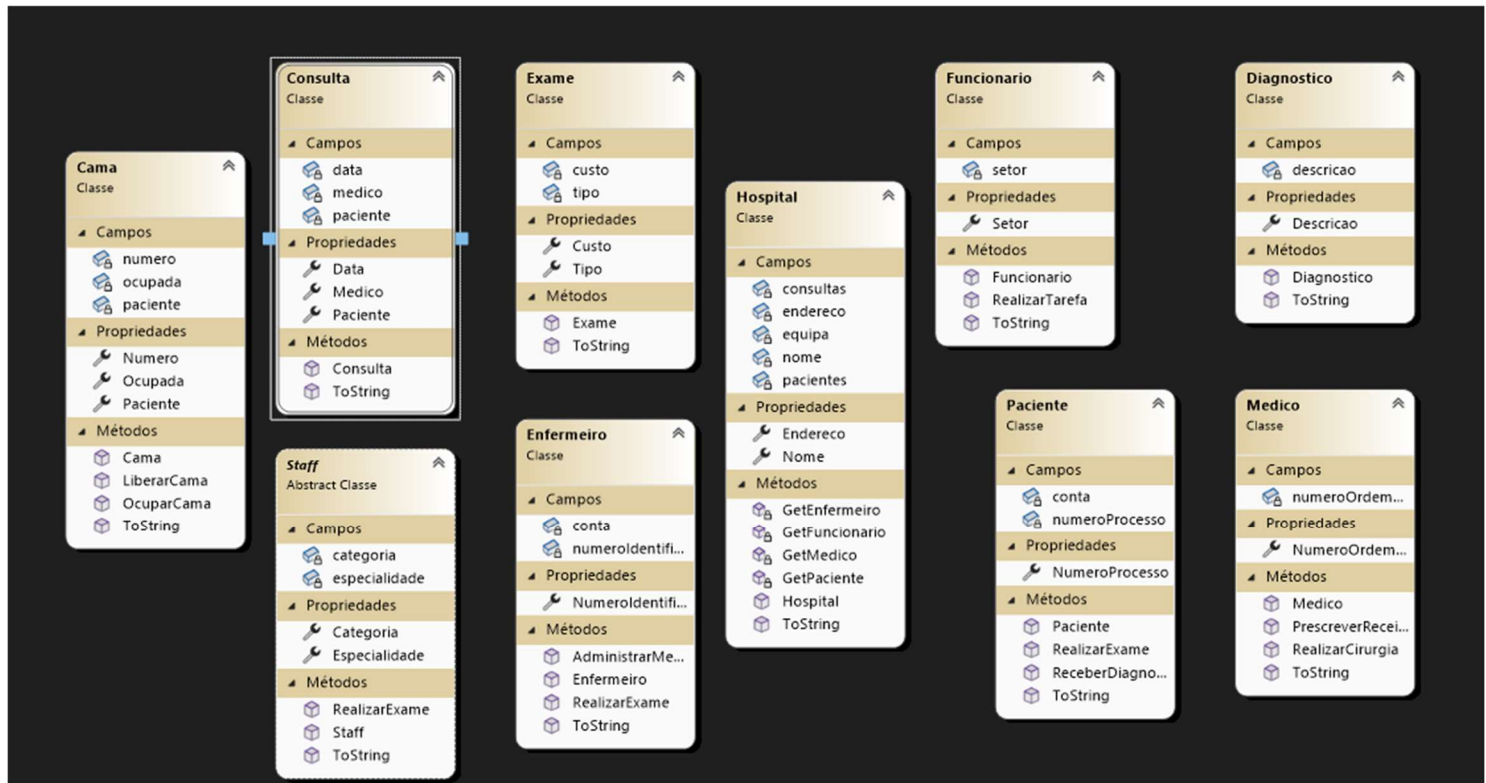
- **Informações do Hospital:**

Apresenta dados gerais sobre o hospital, incluindo nome e endereço, proporcionando uma visão rápida das informações principais.

- **Listar Staff:**

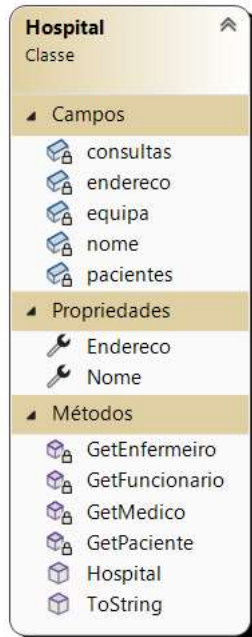
Facilita o acompanhamento e gestão da equipe hospitalar, incluindo médicos, enfermeiros e funcionários, contribuindo para uma administração eficiente.

Diagrama de Classes



Descrição das Classes

Classe Hospital



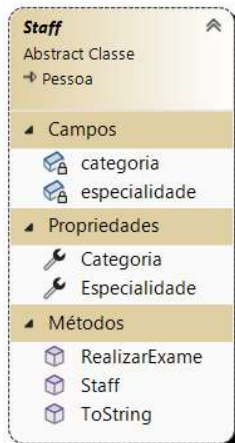
Propriedades:

- Nome (string) – Nome do hospital
- Endereço (string) – Endereço do hospital
- Staff ([] list de staff) – Membros do Staff
- Pacientes ([] list de pacientes) – Pacientes
- Consultas ([] list de consultas) – Consultas

Métodos:

- AdicionarStaff (Staff) – Adiciona um membro da equipe ao hospital.
- ListarStaff () – Retorna uma lista de todos os membros da equipe.
- AdicionarPacientes(Paciente paciente) – Adiciona um paciente ao hospital.
- ListarPaciente() – Retorna uma lista de todos os pacientes no hospital.
- AdicionarConsulta(Consulta consulta) – Adiciona uma consulta ao sistema.
- ListarConsultas() – Retorna uma lista de todas as consultas agendadas.

Classe Staff (Classe Abstrata)



Propriedades:

- Nome (string) – Nome do membro da equipe.
- Categoria (string) – Categoria do membro (médico, enfermeiro, etc.).
- Especialidade (string) – Especialidade do membro (se aplicável).

Métodos:

- RealizarConsulta (Paciente paciente) – Realizar uma consulta com o paciente especificado.
- RealizarExame (Paciente paciente, Exame exame) – Realiza um exame em um paciente.

Classe Paciente



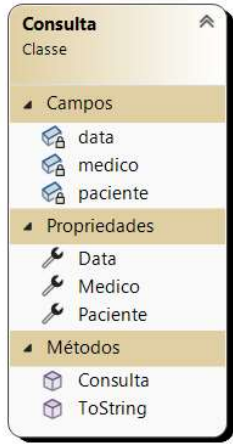
Propriedades:

- Nome (string) -Nome do paciente.
- NumeroCartaoCidado (int) – Número cartão de cidadão.
- Diagnosticos ([] list de Diagnostico) – Diagnóstico do paciente.
- Exames ([] list de Exame) – Exames do paciente.
- CustoTotal (decimal) – Custo total dos tratamentos do paciente.

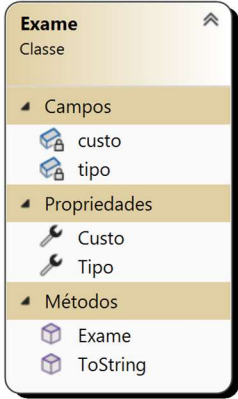
Métodos:

- AdicionarDiagnostico (string diagnostico) – Adiciona um novo diagnóstico ao paciente.
- AdicionarExame (Exame exame) – Adiciona um exame ao histórico do paciente.

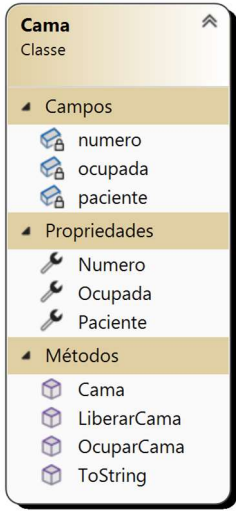
Classe Consulta

 The diagram shows the structure of the 'Consulta' class. It includes fields (data, medico, paciente), properties (Data, Medico, Paciente), and methods (Consulta, ToString).	Propriedades: • Médico (Staff) – Médico responsável pela consulta. • Paciente (Paciente) – Paciente da consulta. • Data (Data Time) – Data e hora da consulta. Métodos: • Agendar Consulta (string Consulta) Marcar consulta nome do medico, nome paciente e data
---	---

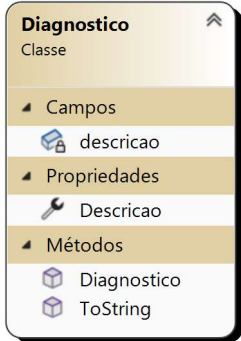
Classe Exame

 The diagram shows the structure of the 'Exame' class. It includes fields (custo, tipo), properties (Custo, Tipo), and methods (Exame, ToString).	Propriedades: • Tipo (string) – Tipo de exame (ex: radiografia, análise de sangue, etc.). • Custo (decimal) – Custo do exame.
---	--

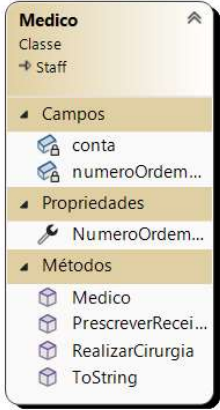
Classe Cama

 Cama Classe Campos - numero - ocupada - paciente Propriedades - Numero - Ocupada - Paciente Métodos - Cama - LiberarCama - OcuparCama - ToString	Propriedades: - Numero (int) – Número da cama. - Ocupada (bool) – Indica se a cama está ocupada ou não. - Paciente (Paciente) – Paciente atualmente utilizando a cama. Métodos: - OcuparCama (Paciente paciente) – Ocupa a cama com o paciente especificado. - LiberarCama () -Libera a cama
---	---

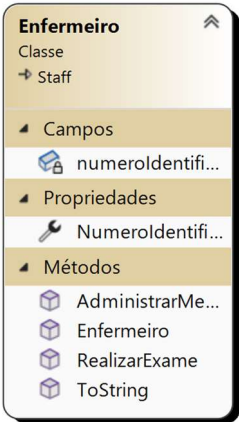
Classe Diagnóstico

 Diagnostico Classe Campos - descricao Propriedades - Descricao Métodos - Diagnostico - ToString	Propriedades: Descricao (string) – Descrição do diagnóstico.
--	--

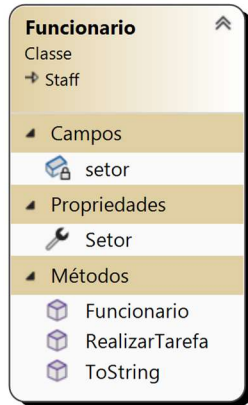
Classe Médico

 The diagram shows the Medico class structure. It is a class (Classe) and a staff member (Staff). It has two fields (Campos): conta and numeroOrdem... It has one property (Propriedades): NumeroOrdem... It has four methods (Métodos): Medico, PrescreverRecei..., RealizarCirurgia, and ToString.	Propriedades: - NumeroIdentificacaoMedico (int) – Número de registo médico. Métodos: - PrescreverMedicamento (Paciente paciente, string medicamento) – Prescreve um medicamento para o paciente.
--	--

Classe Enfermeiro

 The diagram shows the Enfermeiro class structure. It is a class (Classe) and a staff member (Staff). It has one field (Campos): numeroIdentifi... It has one property (Propriedades): NumeroIdentifi... It has four methods (Métodos): AdministrarMe..., Enfermeiro, RealizarExame, and ToString.	Propriedades: - NumeroIdentificacaoEnf (int) – Número de identificação do Enfermeiro no hospital. Métodos: - AdministraraMedicamento (Paciente paciente, string medicamento) – Administra um medicamento ao paciente.
--	---

Classe Funcionário

**Propriedades:**

- Setor (string) – Setor em que o funcionário está alocado.

Métodos:

- RealizarTarefa (string tarefa) – Realiza uma tarefa específica no setor.

Conclusão

O desenvolvimento deste sistema de gestão hospitalar em C# representa uma etapa significativa no meu percurso académico, proporcionando uma oportunidade valiosa para aplicar os conhecimentos adquiridos em Programação Orientada a Objetos (POO) de uma forma prática e relevante. A abordagem modular adotada neste projeto não só reflete os princípios fundamentais da POO, mas também contribui para a flexibilidade e escalabilidade do sistema.

A criação de funcionalidades específicas, como a gestão de pacientes, profissionais de saúde, agendamento de consultas e realização de exames, demonstra o compromisso em abordar os desafios intrínsecos à administração hospitalar. A introdução de listas em substituição de arrays, conforme indicado nas orientações da segunda fase, reflete a adaptação constante do código às boas práticas e à evolução das exigências do projeto.

O desenvolvimento da aplicação permitiu-me compreender a importância da organização do código, a utilização de listas para melhor manipulação de dados e a aplicação eficaz de princípios de encapsulamento e herança. Além disso, a interação com o sistema através de um menu intuitivo e a criação de classes específicas de I/O para cada entidade contribuem para uma experiência de utilizador mais agradável.

Em suma, este projeto não apenas ampliou a minha compreensão prática de C# e POO, mas também evidenciou a capacidade de traduzir conceitos teóricos em soluções concretas. Este trabalho reflete o compromisso com a excelência na implementação de sistemas de software e representa um passo significativo no desenvolvimento das minhas competências como programadora.